

DataHarvest

Rapport technique

Framework de scraping modulaire — Projet final

Romain Pintre • Nassim Boufalous

Mastère Développement, Data & IA — 4^e année
IPSSI Montpellier

31 juillet 2026

1. Introduction et périmètre

Après avoir manipulé **requests** + **BeautifulSoup4**, **Selenium** et **Scrapy** au cours de la formation, ce projet avait pour objectif de concevoir notre propre mini-framework de scraping, DataHarvest, en binôme, en cinq heures. L'exercice ne consistait pas à écrire un énième script de scraping ad hoc, mais à produire un outil **générique et configurable** : un nouveau site doit pouvoir être scrapé en écrivant uniquement un fichier de configuration YAML, sans toucher au code Python.

Quel problème DataHarvest résout-il ? Un script de scraping écrit "à la main" pour un site mélange en général au même endroit la logique HTTP (requêtes, retries, en-têtes), l'extraction (sélecteurs CSS), la validation des données et leur stockage. Changer de site oblige à réécrire le script en entier, et il n'existe aucune garantie que les erreurs réseau ou les données incomplètes soient gérées de façon cohérente d'un script à l'autre. DataHarvest sépare ces quatre responsabilités en composants indépendants, assemblés par un chef d'orchestre (*Orchestrator*), et pilotés entièrement par un fichier de configuration déclaratif.

Limites de portée assumées : DataHarvest ne s'adresse volontairement qu'aux sites HTML statiques accessibles via requests, sans rendu JavaScript (pas de Selenium/Playwright intégré). Il est **synchrone** — une requête HTTP à la fois — ce qui le rend plus simple à lire et à déboguer qu'un framework asynchrone, au prix de performances plus faibles sur de gros volumes (voir section 3). L'extraction repose sur des sélecteurs CSS positionnels : un champ dont la présence varie d'un élément à l'autre sur la même page peut désaligner les données si l'on n'y prête pas attention (voir section 5, difficulté n°2).

Sites choisis pour tester le framework : cinq sites ont été sélectionnés parmi la liste de vingt proposée dans le sujet, en couvrant quatre niveaux de difficulté différents (contrainte du sujet : au moins deux niveaux) : **books.toscrape.com** et **quotes.toscrape.com** (niveau ★, sites d'entraînement), **pypi.org/search** (niveau ★★), **blogdumoderateur.com** (niveau ★★★, média avec pagination) et **news.ycombinator.com** (niveau ★★★★, sélecteurs complexes). Ce choix visait délibérément la diversité des structures HTML (catalogue produit, citations, recherche de paquets, articles de blog, forum de liens) plutôt que des sites tous construits sur le même schéma.

2. Architecture et choix de conception

2.1 Vue d'ensemble

Le framework est découpé en sept modules, chacun avec une responsabilité unique : **Config** (chargement/validation YAML-JSON), **Middleware** (traitements transverses sur les requêtes/réponses HTTP), **Fetcher** (téléchargement HTTP), **Pipeline** (extraction des données), **Validator** (filtrage des éléments invalides), **Store** (persistance multi-backend) et **Orchestrator** (assemblage et pilotage du cycle complet). Une interface en ligne de commande (app.py) expose trois sous-commandes : crawl, export et validate.

```
Config (YAML)
|
v
Orchestrator
|-- Fetcher [+ Middleware chain] --> HTML brut
|-- Pipeline.process(html) -----> list[dict]
|-- Validator.validate(items) ----> items valides / rejetes
'-- Store.save(items) -----> csv / sqlite / json
```

2.2 Pourquoi une architecture en composants découplés plutôt qu'un script monolithique ?

Un script monolithique mélange logique réseau, extraction et stockage dans les mêmes fonctions : il est difficile à tester (impossible de vérifier l'extraction sans faire une vraie requête HTTP), difficile à faire évoluer (ajouter un backend de stockage oblige à relire tout le script) et difficile à lire pour quelqu'un qui découvre le projet. En séparant chaque responsabilité dans son propre module :

- **Testabilité** : chaque composant est testé isolément, avec ses dépendances mockées. Les 44 tests unitaires du projet ne font aucun appel réseau réel — `Fetcher.fetch()` est testé en patchant `requests.Session.get`, `Orchestrator.run()` en patchant `fetcher.fetch` directement.
- **Extensibilité** : ajouter un quatrième backend de stockage (par exemple PostgreSQL) n'impacte que `store.py` ; ajouter un `RateLimitMiddleware` n'impacte que `middleware.py` et la liste passée au constructeur du `Fetcher`.
- **Lisibilité** : chaque fichier fait une seule chose. `orchestrator.py` ne contient aucune ligne de parsing HTML, `pipeline.py` ne contient aucune ligne de gestion de retry.

2.3 Pourquoi BasePipeline est une classe abstraite (ABC) ?

`BasePipeline` définit deux méthodes abstraites, `process(html)` et `next_page_url(html, current_url)`, via le module `abc` de Python. Une classe normale avec des méthodes vides (par exemple `def process(self, html): pass`) laisserait le programme démarrer même si une sous-classe oublie d'implémenter `process()` — l'erreur ne serait découverte qu'au moment de l'appel, potentiellement en production, potentiellement après avoir déjà consommé du temps de scraping réel. Une ABC lève `TypeError` dès l'instanciation (`Pipeline(...)`) si une méthode abstraite n'est pas implémentée. C'est une garantie apportée au niveau du *contrat* de la classe plutôt qu'au niveau de son comportement à l'exécution — l'utilisateur du framework sait immédiatement, avant même de lancer un scraping, si sa pipeline personnalisée est complète.

2.4 Pourquoi le pattern middleware pour le Fetcher ?

Le `Fetcher` doit gérer des préoccupations transverses (journalisation, retry avec backoff exponentiel) qui n'ont rien à voir avec le téléchargement HTTP lui-même et qui peuvent varier indépendamment les unes des autres. Sans le pattern middleware, `Fetcher.fetch()` contiendrait un empilement de `if/else` pour chaque politique de retry envisageable, rendant la méthode de plus en plus difficile à faire évoluer à chaque nouvelle politique. Avec le pattern middleware, chaque middleware implémente `process_request(url, headers)` et `process_response(response)`, et le `Fetcher` se contente de faire passer chaque requête/réponse dans la chaîne, dans l'ordre où les middlewares ont été ajoutés au constructeur — exactement le même pattern que celui utilisé par Django ou Flask pour leurs middlewares HTTP.

Sans ce pattern, gérer le retry aurait nécessité soit de coder la logique de retry directement dans `fetch()` (couplage fort, impossible à désactiver ou remplacer sans modifier le `Fetcher`), soit d'utiliser un décorateur externe appliqué autour de `fetch()` (fonctionnellement proche, mais qui perd l'accès à l'objet `response` complet pour décider dynamiquement si un nouveau essai est nécessaire selon le code HTTP). Le pattern retenu permet d'ajouter un futur `RateLimitMiddleware` (garantissant un délai minimum entre deux requêtes vers le même domaine) sans modifier une seule ligne du `Fetcher`.

2.5 Pourquoi la configuration via YAML plutôt que des arguments CLI ?

Un scraping réel nécessite de nombreux paramètres : l'URL de départ, le pattern de pagination, un sélecteur CSS par champ à extraire (souvent quatre à cinq champs), le délai entre requêtes, le nombre de tentatives, le backend de stockage et son chemin. Passer tout cela en arguments CLI donnerait une commande illisible et source d'erreurs (--selector-titre --selector-url --selector-date --pagination-pattern ...). Le fichier YAML garde cette configuration détaillée dans un document structuré, versionnable dans Git au même titre que le code, et réutilisable sans avoir à retaper la commande. Le CLI garde un rôle de point d'entrée léger (--config chemin.yaml, --dry-run) : on préférerait des arguments CLI seuls pour un outil avec très peu de paramètres ponctuels (par exemple un simple script de vérification d'URL), mais pas pour un outil dont la configuration complète définit le comportement d'un composant entier (les sélecteurs).

2.6 Pourquoi l'injection de dépendances dans le constructeur plutôt que des imports directs ?

Chaque composant reçoit ses dépendances via son constructeur au lieu d'importer directement les classes concrètes dont il a besoin :

```
class Orchestrator:
    def __init__(self, config: Config):
        self.fetcher = Fetcher(config, middlewares=[...])
        self.pipeline = PaginationPipeline(config.selectors, config.pagination, ...)
        self.validator = Validator(required_fields=['titre', 'url'])
        self.store = Store(config.store.backend, config.store.path)
```

Cette approche permet de remplacer une dépendance par un double de test sans modifier le code de la classe qui l'utilise. Exemple concret tiré du projet, tests/test_orchestrator.py :

```
orchestrator = Orchestrator(config)
with patch.object(orchestrator.fetcher, 'fetch', return_value=ONE_PAGE_HTML):
    report = orchestrator.run()
```

Ce test construit un Orchestrator réel, avec un vrai Fetcher, un vrai Pipeline, un vrai Validator et un vrai Store — seule la méthode fetch() de l'instance fetcher déjà construite est remplacée après coup. Sans injection de dépendances (si Orchestrator faisait from .fetcher import Fetcher puis appelait des fonctions de module directement), il aurait fallu patcher la classe Fetcher elle-même de façon plus intrusive, avec le risque d'affecter d'autres tests exécutés dans le même processus. La même approche est utilisée dans tests/test_app_cli.py (écrit par l'étudiant B) pour tester le CLI sans requête réseau, en patchant dataharvest.fetcher.Fetcher.fetch.

3. Comparaison avec Scrapy

3.1 Fonctionnalités de Scrapy absentes de DataHarvest

Fonctionnalité Scrapy	Pourquoi absente de DataHarvest
Requêtes asynchrones/concurrentes (Twisted)	Hors scope pour 5h de développement : une boucle asynchrone correcte (gestion des exceptions par requête, limitation de concurrence par domaine) est un sujet à part entière.
Pipeline d'items configurable en chaîne	DataHarvest a une seule étape de traitement (Validator) entre l'extraction et le stockage ; Scrapy permet de chaîner plusieurs <i>Item Pipelines</i> (nettoyage, enrichissement, déduplication avancée). Complexité jugée disproportionnée pour le périmètre visé.
Extensions (AutoThrottle, HttpCache, cookies avancés)	Fonctionnalités utiles en production mais non essentielles pour démontrer l'architecture ; auraient consommé le temps disponible sans changer la structure du framework.
Export multi-format natif (scrapy crawl -o out.json)	DataHarvest couvre ce besoin via <code>Store.export_to()</code> et la sous-commande <code>export</code> , mais sans la détection automatique de format à la volée que propose Scrapy.
Shell interactif de debug (scrapy shell)	Outil de productivité important pour Scrapy, mais indépendant de l'architecture centrale du framework ; non implémenté faute de temps.

3.2 Dans quels cas préférer DataHarvest à Scrapy ?

- **Script ponctuel et périmètre restreint** : pour scraper un ou deux sites sans installer une dépendance lourde (Scrapy + Twisted), DataHarvest ne nécessite que `requests`, `beautifulsoup4` et `pyyaml`.
- **Besoin de comprendre/modifier chaque étape** : l'architecture de Scrapy (signaux, middlewares Twisted, items) a une courbe d'apprentissage réelle. Un développeur qui doit ajouter une transformation ponctuelle sur les données peut lire et modifier `pipeline.py` de DataHarvest en quelques minutes.

3.3 Impact du synchrone vs asynchrone : calcul théorique

Le sujet demande un calcul théorique du temps de scraping pour 100 pages avec un délai de 1 seconde entre requêtes (`DOWNLOAD_DELAY=1s` côté Scrapy, équivalent à `fetcher.delay: 1` côté DataHarvest).

DataHarvest (synchrone) : chaque page est récupérée l'une après l'autre. En estimant un temps de requête moyen de 0,3 s (réponse réseau + parsing), le temps total est approximativement :

```
100 x (0,3s + 1s) ~= 130 secondes (~2 min 10s)
```

Scrapy (asynchrone, Twisted) : par défaut, Scrapy traite jusqu'à `CONCURRENT_REQUESTS = 16` requêtes en parallèle, tout en respectant `DOWNLOAD_DELAY` par domaine. Dans le meilleur cas théorique (aucun goulot d'étranglement autre que le délai imposé), le temps total tend vers :

```
(100 / 16) x (0,3s + 1s) ~= 8,1 secondes
```

En pratique, l'overhead de planification des requêtes (scheduler, gestion des signaux, sérialisation des items) rapproche ce chiffre de 15 à 20 secondes plutôt que 8. L'écart reste néanmoins d'un ordre de grandeur : sur un volume de pages important, l'architecture asynchrone de Scrapy devient nettement plus rapide que l'approche séquentielle de DataHarvest, ce qui est cohérent avec le choix

délibéré de DataHarvest de privilégier la simplicité du code à la performance brute.

4. Cadre légal et éthique

4.1 Analyse des robots.txt des cinq sites

Les fichiers robots.txt des cinq sites choisis ont été récupérés et lus directement (et non supposés) avant de finaliser les configurations :

Site	robots.txt	URL interdites concernées	Respecté ?
books.toscrape.com	N'existe pas (404)	Aucune (sandbox d'entraînement)	Oui — aucune restriction
quotes.toscrape.com	N'existe pas (404)	Aucune (sandbox d'entraînement)	Oui — aucune restriction
pypi.org	Existe	Disallow: /search* (explicite)	Non — voir ci-dessous
blogdumoderateur.com	Existe	/wp-admin, /category/*/*, /feed/, /comments... — la page d'accueil / n'est pas concernée	Oui
news.ycombinator.com	Existe	Crawl-delay: 30 pour tous les user-agents ; /vote?, /login, etc. (actions, pas la lecture)	Oui, après correction (voir ci-dessous)

Cas non conforme assumé — pypi.org : le robots.txt de PyPI interdit explicitement /search*. La configuration site3.yaml cible pourtant cette URL. Ce choix est volontaire et documenté : le site a par ailleurs mis en place depuis la rédaction du sujet une protection anti-bot serveur (Fastly "Client Challenge" en JavaScript) qui bloque de toute façon totalement la requête avant même d'atteindre le contenu — **aucune donnée personnelle ni aucun contenu n'a donc réellement été collecté** sur ce site. Il est conservé dans le projet comme exemple honnête d'échec réel plutôt que remplacé silencieusement par un site plus simple, conformément à l'esprit de la section 5 du sujet (difficultés rencontrées).

Correction appliquée — news.ycombinator.com : la configuration initiale utilisait un délai de 1,5 s entre requêtes. La vérification du robots.txt a révélé une directive Crawl-delay: 30 explicite. Le délai a été corrigé à 30 secondes dans site5.yaml avant la mise en production de cette configuration, pour respecter la règle réellement déclarée par le site.

4.2 Base légale (RGPD, article 6)

Les données collectées par les cinq configurations sont des données publiques, déjà librement accessibles sans authentification (catalogue de produits, citations publiques, métadonnées de paquets logiciels, articles de presse, discussions publiques sur un forum). La base légale mobilisable pour ce traitement est l'**intérêt légitime** (article 6.1.f du RGPD) : le traitement est réalisé dans un cadre strictement pédagogique, sans finalité commerciale, sans republication des données collectées, et les volumes stockés localement (output) ne sont pas destinés à être diffusés au-delà du dépôt Git du projet.

4.3 Les données collectées sont-elles des données personnelles ?

Le champ auteur de la configuration quotes.toscrape.com contient des noms de personnes (auteurs de citations), mais il s'agit de figures publiques ou historiques citées dans un contexte de citations déjà publiques — pas de profilage, pas de croisement avec d'autres sources, pas de donnée sensible au sens de l'article 9 du RGPD. Les autres configurations ne collectent aucune donnée directement liée à une personne physique identifiable : la configuration news.ycombinator.com extrait volontairement le titre, le score et le nombre de commentaires d'un post, mais pas le pseudonyme de son auteur ni ceux des commentateurs.

4.4 Mécanismes techniques de "bon citoyen du web"

- **User-Agent identifiable** : chaque configuration déclare DataHarvest/1.0 (+r.pintre@gmail.com), permettant à l'exploitant du site de contacter l'auteur du trafic en cas de problème.
- **Throttling adapté par site** : le délai entre requêtes (fetcher.delay) est ajusté selon le Crawl-delay réellement déclaré (30 s pour Hacker News, 1 à 1,5 s pour les autres sites).
- **Retry avec backoff exponentiel** : RetryMiddleware espace ses nouvelles tentatives ($\text{base_delay} * 2^{\text{tentative}}$) plutôt que de marteler un serveur qui répond déjà en erreur (429, 5xx).
- **Déduplication systématique** (UNIQUE(url) en sqlite, vérification par url en csv/json) : évite de retraiter inutilement une ressource déjà collectée lors d'exécutions répétées.

5. Difficultés et rétrospective

5.1 Difficulté technique principale

La difficulté la plus significative n'a pas été l'écriture d'un composant isolé, mais l'**alignement positionnel des champs extraits par la Pipeline**. La conception retenue détermine le nombre d'items d'une page à partir du **champ primaire** (la première clé déclarée dans selectors), puis associe chaque champ suivant par index. Cette approche fonctionne bien tant que chaque champ apparaît exactement une fois par item — mais deux cas réels l'ont mise en défaut :

- Sur quotes.toscrape.com, une citation peut avoir plusieurs tags. Utiliser le sélecteur qui remonte le plus d'éléments comme référence de comptage (première implémentation) faisait exploser le nombre d'"items" détectés et désalignait tous les autres champs. **Résolu** en basant le comptage sur le champ primaire (titre) plutôt que sur le sélecteur le plus fourni, et en restreignant le sélecteur de tag à `a.tag:first-of-type` pour ne garder qu'un tag par citation.
- Sur blogdumoderateur.com, seuls 12 des 36 articles de la page d'accueil affichent une étiquette de catégorie visible. Inclure ce champ dans les sélecteurs aurait décalé les données des articles suivants. **Résolu** en omettant volontairement ce champ pour ce site (le sujet ne l'impose pas comme champ obligatoire, seuls titre et url le sont).

Un second problème lié à l'alignement a affecté quotes.toscrape.com spécifiquement : ce site ne fournit aucune URL propre par citation, seulement un lien partagé vers la page de l'auteur. Comme le Store déduplique par url (comportement voulu pour éviter les vrais doublons), plusieurs citations distinctes du même auteur se faisaient silencieusement écraser les unes les autres. **Résolu** par une désambiguïsation interne à la Pipeline : lorsqu'une URL identique apparaît plusieurs fois dans le même traitement, un fragment `#N` lui est ajouté pour la rendre unique sans en changer la cible réelle.

5.2 Autres bugs corrigés pendant le développement

- **Fuite de connexions sqlite** : `with sqlite3.connect(...)` as `conn`: ne ferme pas la connexion en Python — le context manager de `sqlite3.Connection` gère uniquement le `commit/rollback`, pas `close()`. Détecté en exécutant les tests avec les `ResourceWarning` promus en erreurs. Corrigé par un `try/finally` explicite à chaque connexion.
- **Incompatibilité entre composants écrits séparément par les deux membres du binôme** : le CLI (étudiant B) appelait `Orchestrator.run(dry_run=args.dry_run)`, mais `Orchestrator.run()` (étudiant A) ne supportait pas ce paramètre. L'erreur (`TypeError`) a été détectée en exécutant la suite de tests complète après intégration du commit du CLI — révélée par le propre test de l'étudiant B (`test_cmd_crawl_dry_run`). Corrigée en ajoutant le mode `dry_run` à `Orchestrator.run()` : il `fetch/parse` uniquement la première page, affiche les items, ne stocke rien.
- **`fetch_all()` n'utilisait pas `config.fetcher.delay`** : un délai aléatoire (`random.uniform(2, 5)`) était codé en dur, en contradiction avec l'exigence explicite du sujet (section 4.3). Repéré lors d'une relecture croisée du code, corrigé pour utiliser le délai configuré.

5.3 Ce qui serait fait différemment

Avec plus de temps, la Pipeline gagnerait à scoper l'extraction secondaire au conteneur de chaque item (par exemple en retrouvant l'ancêtre commun du champ primaire) plutôt que de se reposer sur un alignement positionnel global — cela aurait évité une bonne partie du travail de contournement décrit en 5.1, au prix d'une implémentation plus complexe qui n'était pas nécessaire pour les cinq sites effectivement traités.

5.4 Fonctionnalité non implémentée faute de temps

Le `RateLimitMiddleware` (bonus, section 3.2 du sujet), qui garantirait un délai minimum entre deux requêtes vers un même domaine indépendamment de `fetcher.delay`, n'a pas été implémenté. Le pattern middleware déjà en place permettrait de l'ajouter sans modifier le `Fetcher` (voir section 2.4) — il suffirait d'implémenter la classe et de l'ajouter à la liste passée au constructeur.

5.5 Répartition des tâches et visibilité dans Git

La répartition suit globalement le découpage du sujet : l'étudiant A (Romain Pintre) a écrit `Config`, `Pipeline`, `Store`, `Orchestrator`, les cinq configurations de sites et la quasi-totalité des tests unitaires (y compris ceux du `Middleware` de l'étudiant B, pour compléter sa couverture) ; l'étudiant B (Nassim Boufalous) a écrit `Middleware`, `Fetcher`, `Validator`, le `CLI` et le `README`. Cette répartition est entièrement visible dans l'historique Git : chaque commit porte le nom de son auteur réel, et les deux bugs d'intégration décrits en 5.2 sont eux-mêmes des commits distincts ("fix: ...") situés au point de jonction entre le travail des deux, après que chacun a intégré les commits de l'autre.

6. Perspectives

6.1 Une sixième brique : Notifier

Un module Notifier viendrait s'ajouter à l'Orchestrator pour alerter automatiquement lorsqu'un scraping produit anormalement peu de résultats — signe fréquent d'un changement de structure HTML côté site cible (exactement le type de panne rencontrée avec PyPI, section 4.1). Interface envisagée :

```
class Notifier:
    def __init__(self, webhook_url: str, min_items_threshold: int = 1):
        self.webhook_url = webhook_url
        self.min_items_threshold = min_items_threshold

    def notify(self, report: dict) -> None:
        """Envoie une alerte si items_stockes passe sous le seuil."""
        if report['items_stockes'] < self.min_items_threshold:
            requests.post(self.webhook_url, json={'text': f'Alerte: {report}'})
```

L'Orchestrator appellerait `notifier.notify(report)` à la fin de `run()`, en respectant le même principe d'injection de dépendances que les autres composants (le Notifier serait optionnel, passé au constructeur).

6.2 Compatibilité avec les sites JavaScript sans Selenium

Plutôt que d'intégrer Selenium (lourd, lent à démarrer), deux pistes plus légères : **Playwright**, dont l'API asynchrone native et le mode headless démarrent nettement plus vite que Selenium/WebDriver, pourrait remplacer le Fetcher pour les sites qui l'exigent, sans changer la Pipeline (le HTML rendu resterait passé à `process()` de la même façon) ; ou un service de pré-rendu tiers (Splash, Prerender.io) interrogé comme un proxy HTTP classique, ce qui ne nécessiterait aucune modification du Fetcher au-delà de l'URL de base.

6.3 Distribution sur PyPI

Le projet a déjà une structure proche de celle attendue pour un paquet publiable : `__version__` déclaré dans `dataharvest/__init__.py`, point d'entrée CLI via `__main__.py`. Il manquerait un `pyproject.toml` avec les métadonnées du paquet (classifiers, dépendances déclarées à partir de `requirements.txt`), et une intégration continue (GitHub Actions) qui exécute la suite de tests puis publie automatiquement sur un tag de version.

Bibliographie / Sources

- Sujet du projet — *DataHarvest, Framework de scraping modulaire — Projet final*, IPSSI Montpellier, Mastère Développement, Data & IA.
- Documentation officielle Python — module `abc` (classes abstraites) : docs.python.org/3/library/abc.html
- Documentation officielle Python — module `sqlite3`, comportement du context manager (commit/rollback uniquement, pas de fermeture automatique de connexion) : docs.python.org/3/library/sqlite3.html

- Documentation Scrapy — architecture, CONCURRENT_REQUESTS, DOWNLOAD_DELAY : docs.scrapy.org
- RGPD — Règlement (UE) 2016/679, article 6 (licéité du traitement) et article 9 (catégories particulières de données) : eur-lex.europa.eu
- Fichiers robots.txt consultés directement : books.toscrape.com/robots.txt, quotes.toscrape.com/robots.txt, pypi.org/robots.txt, blogdumoderateur.com/robots.txt, news.ycombinator.com/robots.txt
- Documentation Playwright (pour la section Perspectives) : playwright.dev/python